

Skills

Package reusable knowledge into Markdown files and inject them into any agent's context automatically.

Overview

A skill is a directory containing a `SKILL.md` file. When an agent runs, GoClaw reads the skill files that are in scope and injects their content into the system prompt under an `## Available Skills` section. The agent then uses that knowledge without you having to repeat it in every conversation.

Skills are useful for encoding recurring procedures, tool usage guides, domain knowledge, or coding conventions that the agent should always follow.

SKILL.md Format

Each skill lives in its own directory. The directory name is the skill's **slug** — the unique identifier used for filtering and search.

```
~/goclaw/skills/  
└─ code-reviewer/  
   └─ SKILL.md
```

A `SKILL.md` file has an optional YAML frontmatter block followed by the skill content:

```
---  
name: Code Reviewer  
description: Guidelines for reviewing pull requests – style, security, and performance check  
---  
  
## How to Review Code  
  
When asked to review code, always check:  
  
1. Security – SQL injection, XSS, hardcoded secrets  
2. Error handling – all errors returned or logged  
3. Tests – new logic has corresponding test coverage  
  
Use `{baseDir}` to reference files alongside this SKILL.md:  
- Checklist: {baseDir}/review-checklist.md
```

The `{baseDir}` placeholder is replaced at load time with the absolute path to the skill directory, so you can reference companion files.

Multiline block : YAML frontmatter supports multiline strings for `description` using the `|` block scalar. This is useful for longer skill descriptions without hitting YAML line limits.

Frontmatter fields:

Field	Description
<code>name</code>	Human-readable display name (defaults to directory name)
<code>description</code>	One-line summary used by <code>skill_search</code> to match queries

6-Tier Hierarchy

GoClaw loads skills from six locations in priority order. A skill in a higher-priority location overrides one with the same slug from a lower one:

Priority	Location	Source label
1 (highest)	<code><workspace>/skills/</code>	<code>workspace</code>
2	<code><workspace>/.<code>agents</code>/skills/</code>	<code>agents-project</code>
3	<code>~/.<code>agents</code>/skills/</code>	<code>agents-personal</code>
4	<code>~/.<code>goclaw</code>/skills/</code>	<code>global</code>
5	<code>~/.<code>goclaw</code>/skills-store/</code> (DB-seeded, versioned)	<code>managed</code>
6 (lowest)	Built-in (bundled with binary)	<code>builtin</code>

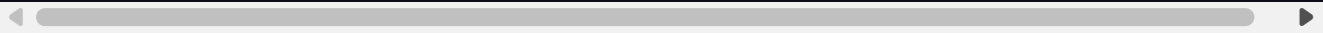
Skills uploaded via the Dashboard are stored in `~/.goclaw/skills-store/` using a versioned subdirectory structure (`<slug>/<version>/SKILL.md`). They act at the `managed` level — above builtin but below the four file-system tiers. The loader always serves the highest-numbered version for each slug.

Precedence example: if you have a `code-reviewer` skill in both `~/.goclaw/skills/` and `<workspace>/skills/` , the workspace version wins.

Hot Reload

GoClaw watches all skill directories with `fnotify` . When you create, modify, or delete a `SKILL.md` , changes are picked up within 500 ms — no restart required. The watcher bumps an internal version counter; agents compare their cached version on each request and reload skills if the counter changed.

```
# Drop a new skill in place – agents pick it up on the next request
mkdir ~/.goclaw/skills/my-new-skill
echo "---\nname: My Skill\ndescription: Does something useful.\n---\n\n## Instructions\n..."
> ~/.goclaw/skills/my-new-skill/SKILL.md
```



Uploading via Dashboard

Go to **Skills** → **Upload** and drop a ZIP file. The ZIP can contain a **single skill** or **multiple skills** in one archive:

```
# Single skill – SKILL.md at root
my-skill.zip
└─ SKILL.md

# Single skill – wrapped in one directory
my-skill.zip
└─ code-reviewer/
   └─ SKILL.md
   └─ review-checklist.md

# Multi-skill ZIP – multiple skills in one upload
skills-bundle.zip
└─ skills/
   └─ code-reviewer/
      └─ SKILL.md
      └─ metadata.json
   └─ sql-style/
      └─ SKILL.md
      └─ metadata.json
```

Uploaded skills are stored in a versioned subdirectory structure under the managed skills directory (`~/.goclaw/skills-store/` by default):

```
~/.goclaw/skills-store/<slug>/<version>/SKILL.md
```

Metadata (name, description, visibility, grants) lives in PostgreSQL; file content lives on disk. GoClaw always serves the highest-numbered version. Old versions are kept for rollback.

Skills uploaded via the Dashboard start with **internal** visibility — immediately accessible to any agent or user you grant access to.

Importing via API

The `POST /v1/skills/import` endpoint accepts the same ZIP format as the Dashboard upload and supports both single and multi-skill archives.

Standard import (JSON response):

```
curl -X POST http://localhost:8080/v1/skills/import \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@skills-bundle.zip"
```

Returns a `SkillsImportSummary` JSON object:

```
{
  "skills_imported": 2,
  "skills_skipped": 0,
  "grants_applied": 3
}
```

Streaming import with SSE progress (`?stream=true`):

```
curl -X POST "http://localhost:8080/v1/skills/import?stream=true" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: text/event-stream" \
  -F "file=@skills-bundle.zip"
```

With `?stream=true`, the server sends Server-Sent Events (SSE) as each skill is processed:

```
event: progress
data: {"phase":"skill","status":"running","detail":"code-reviewer"}

event: progress
data: {"phase":"skill","status":"done","detail":"code-reviewer"}

event: complete
data: {"skills_imported":2,"skills_skipped":0,"grants_applied":3}
```

Hash-based idempotency: The upload endpoint uses a SHA-256 hash of the `SKILL.md` content for deduplication. If the same `SKILL.md` content is uploaded again (even packaged in a different ZIP), no new version is created — the existing version is kept unchanged. Only changes to the actual `SKILL.md` content trigger a new version.

Runtime Environment

Skills that use Python or Node.js run inside a Docker container with pre-installed packages.

Pre-installed Packages

Category	Packages
Python	<code>pypdf</code> , <code>openpyxl</code> , <code>pandas</code> , <code>python-pptx</code> , <code>markdown</code>
Node.js (global npm)	<code>docx</code> , <code>pptxgenjs</code>
System tools	<code>python3</code> , <code>nodejs</code> , <code>pandoc</code> , <code>gh</code> (GitHub CLI)

Writable Runtime Directories

The container root filesystem is read-only. Agents install additional packages to writable volume-backed directories:

```
/app/data/.runtime/
├─ pip/           ← PIP_TARGET (Python packages)
├─ pip-cache/     ← PIP_CACHE_DIR
└─ npm-global/    ← NPM_CONFIG_PREFIX (Node.js packages)
```

Packages installed at runtime persist across tool calls within the same container lifecycle.

Security Constraints

Constraint	Detail
<code>read_only: true</code>	Container rootfs is immutable; only volumes are writable
<code>/tmp</code> is <code>noexec</code>	Cannot execute binaries from tmpfs
<code>cap_drop: ALL</code>	No privilege escalation
Exec deny patterns	Blocks <code>curl</code> <code>sh</code> , reverse shells, crypto miners
<code>.goclaw/</code> denied	Exec tool blocks access to <code>.goclaw/</code> except <code>.goclaw/skills-store/</code>

What Agents Can/Cannot Do

Agents **can**: run Python/Node scripts, install packages via `pip3 install` or `npm install -g` , access files in `/app/workspace/` including `.media/` .

Agents **cannot**: write to system paths, execute binaries from `/tmp` , run blocked shell patterns (network tools, reverse shells).

Bundled Skills

GoClaw ships five core skills bundled inside the Docker image at `/app/bundled-skills/`. They are lowest priority — user-uploaded skills override them by slug.

Skill	Purpose
<code>pdf</code>	Read, create, merge, split PDFs
<code>xlsx</code>	Read, create, edit spreadsheets
<code>docx</code>	Read, create, edit Word documents
<code>pptx</code>	Read, create, edit presentations
<code>skill-creator</code>	Create new skills

Bundled skills are seeded into PostgreSQL on every gateway startup (hash-tracked, no re-import if unchanged). They are tagged `is_system = true` and `visibility = 'public'`.

Dependency System

GoClaw auto-detects and installs missing skill dependencies:

Scanner — statically analyzes `scripts/` subdirectory for Python (`import X`, `from X import`) and Node.js (`require('X')`, `import from 'X'`) imports

Checker — verifies each import resolves at runtime via subprocess (`python3 -c "import X"` / `node -e "require.resolve('X')"`)

Installer — installs by prefix:

Prefix	Effect
<code>pip:name</code>	<code>pip3 install</code> (Python package)
<code>npm:name</code>	<code>npm install -g</code> (Node.js package)
<code>system:name</code>	<code>apk add</code> via pkg-helper (system package)
<code>github:owner/repo[@tag]</code>	GitHub Releases installer — admin-only, SHA256-verified, ELF-validated. Binary lands in <code>/app/data/.runtime/bin/</code> (on <code>\$PATH</code>).

Example SKILL.md frontmatter using `github: :`

```
---
name: my-skill
description: Does things using ripgrep and gh CLI.
deps:
  - github:BurntSushi/ripgrep@14.1.0
  - github:cli/cli@v2.40.0
```

```
- pip:requests
---
```

The `github:` installer fetches the release from GitHub Releases, auto-selects the `linux` + arch-matching asset (amd64 / arm64), verifies SHA256 if the publisher ships `checksums.txt`, validates ELF magic bytes, and extracts to `/app/data/.runtime/bin/`. If no `@tag` is specified, the latest release is used.

Dep checks run in a background goroutine at startup (non-blocking). Skills with missing deps are archived automatically; they are re-activated after deps are installed. You can also trigger a rescan via **Skills** → **Rescan Deps** in the Dashboard or `POST /v1/skills/rescan-deps`.

Built-in Skill Tools

GoClaw provides three built-in tools that agents use to discover and activate skills at runtime.

skill_search

Agents search skills using `skill_search`. The search uses a **BM25 index** built from each skill's name and description, with optional hybrid search (BM25 + vector embeddings) when an embedding provider is configured.

```
# The agent calls this tool internally – you don't call it directly
skill_search(query="how to review a pull request", max_results=5)
```

The tool returns ranked results with name, description, location path, and score. After receiving results, the agent calls `use_skill` then `read_file` to load the skill content.

The index is rebuilt whenever the loader's version counter is bumped (i.e., after any hot-reload event or startup).

use_skill

A lightweight observability marker tool. The agent calls `use_skill` before reading a skill's file, so skill activation is visible in traces and real-time events. It does not load any content itself.

```
use_skill(name="code-reviewer")
# then:
read_file(path="/path/to/code-reviewer/SKILL.md")
```

publish_skill

Agents can register a local skill directory into the system database using `publish_skill`. The directory must contain a `SKILL.md` with a `name` in its frontmatter. The skill is automatically granted to the calling agent after publishing.

```
publish_skill(path="./skills/my-skill")
```

The skill is stored with **private** visibility and auto-granted to the calling agent. Admins can later grant it to other agents or promote visibility via the Dashboard or API.

Granting Skills to Agents (Managed Mode)

Skills published via `publish_skill` start with **private** visibility. Skills uploaded via the Dashboard start with **internal** visibility. Either way, you must **grant** a skill to an agent before it is injected into that agent's context.

Via Dashboard

Go to **Skills** in the sidebar

Click the skill you want to grant

Under **Agent Grants**, select the agent and click **Grant**

The skill is now injected into that agent's context on the next request

To revoke, toggle off the agent in the grants list.

Via API

Grant a skill to an agent:

```
curl -X POST http://localhost:8080/v1/skills/{id}/grants/agent \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"agent_id": "AGENT_UUID", "version": 1}'
```

Revoke an agent grant:

```
curl -X DELETE http://localhost:8080/v1/skills/{id}/grants/agent/{agent_id} \
-H "Authorization: Bearer $TOKEN"
```

Grant a skill to a specific user (so it appears in their agent sessions):

```
curl -X POST http://localhost:8080/v1/skills/{id}/grants/user \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"user_id": "user@example.com"}'
```

Revoke a user grant:

```
curl -X DELETE http://localhost:8080/v1/skills/{id}/grants/user/{user_id} \
-H "Authorization: Bearer $TOKEN"
```

Visibility Levels

Level	Who can access
private	Only the skill owner (uploader)
internal	Agents and users explicitly granted access
public	All agents and users

Examples

Workspace-scoped SQL style guide

```
my-project/
├─ skills/
│   └─ sql-style/
│       └─ SKILL.md
```

```
---
name: SQL Style Guide
description: Team conventions for writing PostgreSQL queries in this project.
---

## SQL Conventions

- Use `$1, $2` positional parameters – never string interpolation
- Always use `RETURNING id` on INSERT
- Table and column names: snake_case
- Never use `SELECT *` in application queries
```

Global "be concise" reminder

```
~/.goclaw/skills/
├─ concise-responses/
│   └─ SKILL.md
```

```
---
name: Concise Responses
description: Keep all responses short, bullet-pointed, and actionable.
---
```

Always:

- Lead with the answer, not the explanation
- Use bullet points for lists of 3 or more items
- Keep code examples under 20 lines

Agent Injection Thresholds

GoClaw decides whether to embed skills inline in the system prompt or fall back to `skill_search` :

Condition	Mode
<code>≤ 40 skills</code> AND estimated tokens <code>≤ 5000</code>	Inline — skills injected as XML in system prompt
<code>> 40 skills</code> OR estimated tokens <code>> 5000</code>	Search — agent uses <code>skill_search</code> tool instead

Token estimate: $(\text{len}(\text{name}) + \text{len}(\text{description}) + 10) / 4$ per skill (~100–150 tokens each).

Disabled skills (`enabled = false`) are excluded from both inline and search injection.

Listing Archived Skills

Skills with missing dependencies are set to `status = 'archived'` and are still visible in the Dashboard. You can list them via `GET /v1/skills?status=archived` or the `skills.list` WebSocket RPC method (which returns `enabled` , `status` , and `missing_deps` fields for each skill).

Skill Evolution

When `skill_evolve` is enabled in agent config, agents gain a `skill_manage` tool that allows them to create, update, and version skills from within conversations — a learning loop where the agent improves its own knowledge base. When `skill_evolve` is **off** (the default), the `skill_manage` tool is hidden from the LLM's tool list entirely.

See [Agent Evolution](#) for full details on the `skill_manage` tool and the evolution workflow.

Common Issues

Issue	Cause	Fix
Skill not appearing in agent	Wrong directory structure (SKILL.md not inside a subdirectory)	Ensure path is <code><skills-dir>/<slug>/SKILL.md</code>
Changes not picked up	Watcher not started (non-Docker setups)	Restart GoClaw; verify <code>skills watcher started</code> in logs
Lower-priority skill used instead of yours	Name collision — slug exists at a higher tier	Use a unique slug, or place your skill at a higher-priority location
<code>skill_search</code> returns no results	Index not built yet (first request) or no description in frontmatter	Add a <code>description</code> to frontmatter; index rebuilds on next hot-reload
ZIP upload fails	No <code>SKILL.md</code> found in ZIP	Place <code>SKILL.md</code> at ZIP root, inside one top-level directory, or use the multi-skill <code>skills/<slug>/SKILL.md</code> layout